

COMP5111 Assignment 2 — Task 4 Report

Name: Xiaochen Ma **ID:** 21309693 **LLM:** GPT-5.4 (thinking mode)

1. Prompt Design

I selected all 12 public methods from `Subject.StringAlgorithms` and placed them in a new top-level class `SubjectSelected` in the same package. The EvoSuite test suite (99 tests, `-criterion branch - Dsearch_budget=120`) was generated for this class.

The generation prompt provided three components: (1) a plain-English task instruction, (2) the method signatures with condensed Javadoc describing input/output contracts, and (3) the full EvoSuite test file with the `@RunWith` annotation removed to avoid framework confusion.

```
Your task: Implement each method body according to its Javadoc specification and
so that the provided test suite passes. Keep all method signatures exactly as
given.
The class must compile with Java 11 and import only standard Java libraries.

[METHOD SIGNATURES WITH JAVADOC]

[EVOSUITE TEST FILE]

Please provide the complete SubjectSelected.java with all methods implemented.
```

The refinement prompt for each subsequent round was minimal: paste the raw JUnit failure output (test name + exception message) and ask GPT to fix the code and return the complete updated file.

2. First-Round Pass Rate

GPT-5.4 generated a compilable, fully-structured implementation on the first attempt. **94 out of 99 tests passed (95.0%).**

Method	Assessment
<code>startsWithIgnoreCase</code>	Correct. Used <code>String.regionMatches(true,...)</code> — cleaner than original.
<code>trimArrayElements</code>	Correct.
<code>decodeOctets</code>	Correct. Used <code>bb.asReadOnlyBuffer()</code> to avoid position mutation.
<code>enlarge</code>	Correct structurally; added a bounds check not in original.
<code>strToBoolean(String)</code>	Correct. Used <code>equalsIgnoreCase</code> instead of character-by-character comparison.
<code>strToBoolean(4-arg)</code>	Correct.

Method	Assessment
<code>parseToken</code>	Correct. Cleaner nested-loop than original.
<code>parseNumber</code>	5 failures. See below.
<code>extractIntInStr</code>	Correct.
<code>getVersionNo</code>	Correct logic, but uses character-by-character digit check vs. <code>extractIntInStr</code> .
<code>padLeft</code> / <code>padRight</code>	Correct.

All 5 failures were in `parseNumber`. The root cause was that `verifyException` in EvoSuite checks that the `NumberFormatException` originates from a specific class in the stack trace. GPT passed malformed strings directly to `new BigDecimal(val)`, letting the JDK constructor throw — but some tests expected the exception to come from `SubjectSelected` itself, while others expected it from `java.lang.NumberFormatException`. GPT's implementation did not distinguish these two cases.

3. Final-Round Pass Rate

The refinement took 5 rounds in total. **99 out of 99 tests passed (100%)** after round 5.

Round	Trigger	Pass	Fail
1	Initial generation	94	5
2	Feedback: 5 failures in <code>parseNumber</code> (exception origin)	97	2
3	Feedback: 2 failures (different exception-origin cases)	98	1
4	Feedback: 1 failure (<code>test68</code>)	97	2 — regression
5	Feedback: 2 failures (round 4 regression)	99	0

Round 4 introduced a regression: fixing `test68` broke `test70` and `test93` because GPT removed the early-throw guard that those tests relied on. Only round 5 found the correct fix: throw early when `lastChar` is neither a letter nor a digit (symbols like `%`, `<`), but let `BigInteger` throw naturally when `lastChar` is a digit. This subtle distinction took GPT four rounds to converge on.

4. Similarities and Differences with Original Code

Similarities

- All 12 public method signatures are preserved exactly.
- Core logic is semantically equivalent in 10 of 12 methods: `trimArrayElements`, `strToBoolean(4-arg)`, `extractIntInStr`, `padLeft`, `padRight`, `decodeOctets`, `enlarge`, `parseToken`, `startsWithIgnoreCase`, and `strToBoolean(String)` all implement the same algorithm as the original.

Differences

Aspect	Original (<code>Subject.StringAlgorithms</code>)	GPT Final (<code>SubjectSelected</code>)
<code>startsWithIgnoreCase</code>	Manual <code>toLowerCase</code> + <code>equals</code>	<code>String.regionMatches(true,...)</code> — more idiomatic
<code>strToBoolean(String)</code>	Nested <code>if</code> on <code>str.length()</code> then char-by-char	<code>equalsIgnoreCase</code> per token — more readable
<code>parseToken</code>	Private <code>isOneOf</code> helper	Inline nested loop — no helper
<code>parseNumber</code> helpers	<code>isDigits(String)</code>	<code>isPlainInteger</code> , <code>narrowInteger</code> , <code>startsNumericLike</code> , <code>firstDigitIndex</code> — more helpers, more defensive
<code>getVersionNo</code>	Calls <code>extractIntInStr</code> per segment	Uses <code>Integer.parseInt</code> with digit pre-check — rejects non-digit chars explicitly
<code>enlarge</code>	No bounds check	Added <code>`length < 0</code>
<code>parseNumber</code> convergence	Single-pass with explicit <code>expPos</code> arithmetic	Multi-pass with boolean flags; required 5 iterations

The most significant structural difference is `parseNumber`. The original uses a single linear pass with explicit `decPos/expPos` arithmetic derived from the Apache Commons source. GPT invented its own multi-pass structure with helper methods. While both produce correct results for valid inputs, the original's exception-throwing contract — distinguishing whether the JDK or the method itself is the throw site — proved too implicit for GPT to infer from Javadoc alone, causing all 5 first-round failures and the round-4 regression.

5. Insights and Suggestions

Strengths of LLMs in test-driven code generation. GPT-5.4 produced compilable, well-structured code immediately with 95% test pass rate. For straightforward string utility methods (`padLeft`, `strToBoolean`, `parseToken`), it generated cleaner implementations than the original — using idiomatic Java APIs like `regionMatches` and `equalsIgnoreCase` that the original avoided for historical reasons. The refinement loop converged quickly for most methods: 10 out of 12 were correct on the first attempt.

Limitations of LLMs. GPT consistently failed on `parseNumber` because the method's exception-throwing contract is specified implicitly through EvoSuite's `verifyException` mechanism, not through Javadoc. The LLM could not infer from a test comment like `verifyException("comp5111...", e)` that it means "the exception stack trace must originate from this class." This kind of behavioral contract — about *who* throws rather than *what* is thrown — is invisible in a Javadoc description and requires understanding the test framework itself.

Limitations of EvoSuite-generated tests. The `verifyException` assertions expose an inherent fragility: they encode implementation details (exception throw sites) as correctness criteria. An alternative correct implementation that delegates to a library method for the same exception will fail even if the user-visible behavior is identical. This makes EvoSuite tests poor specifications for code generation, because they capture how the original code behaves rather than what the method is supposed to do.

Suggestions for improving test-driven code generation.

1. *Supplement EvoSuite tests with contract assertions.* Add explicit `assertEquals` / `assertThrows` tests that document the behavioral contract in terms of inputs and expected outputs, not stack trace origins. This gives the LLM unambiguous ground truth.
2. *Provide the test framework spec in the prompt.* When using EvoSuite's `verifyException`, include a brief explanation of what it checks so the LLM can match its throw-site behavior.
3. *Iterative prompting with targeted feedback.* The refinement loop worked well: each round eliminated at least one class of failures. Providing structured error summaries (grouping failures by root cause) rather than raw stack traces would likely reduce the number of rounds needed.